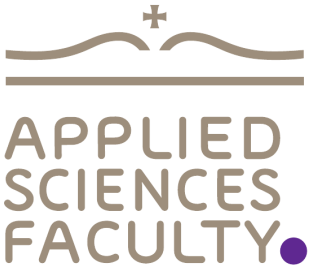
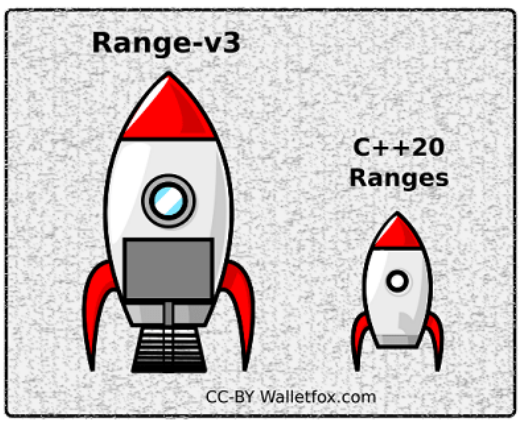
C++ – Ranges, C++23.





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

- В стандарт C++, починаючи з C++20, потроху додають підтримку **діапазонів** (ranges).
- Ідея базується на розробці Eric Niebler, бібліотеці ranges-v3[1]: `https://github.com/ericniebler/range-v3`.
- Підтримка в C++20 мінімальна – більшості корисних елементів немає. В C++23 розширили, але підтримка компіляторами поки не повна.
- Тому виклад далі базуватиметься на ranges-v3, із уточненням щодо підтримуваного в C++23, без диференціації з попереднім стандартом.



[1] boost::ranges – ідейно схожа бібліотека, яка, однак, з 2010 не розвивається. Вона відіграла певну роль у розвитку std::ranges. Цитуючи оригінальну пропозицію Ніблера до стандарту: "*It draws inspiration from the Boost.Range[4] library, the range algorithms in Adobe Source Libraries[1], Elements of Programming by Stepanov and McJones (2009) [20], and from N3351 "A Concept Design for the STL" by Stroustrup and Sutton (2012) [21].*".

Ідея

- Бібліотека реалізовує філософію, схожу на функціональне програмування[2].
  - Вона покладається на концепції із C++20.
- Ключова концепція – Range.
  - Це будь-що, що має методи begin() та end(), які повертають щось ітераторoподібне – що можна використати для пересування по тому діапазону і звертання до його елементів.
- Всі стандартні контейнери та std::string автоматично є діапазонами.
- Бібліотека вважає, що сама концепція ітератора – занадто низькорівнева.
  - Пара ітераторів (інтервал) задає Range (діапазон), однак, в більшості випадків програміст працює саме із діапазоном – ітератори як такі його не цікавлять.

[2] А також, має схожість із list comprehension Python.

Ідея

- Крім того, алгоритми важко комбінувати між собою. Приклад – немає transform_if, а скомбінувати transform і copy_if можна, однак – вимагає зусиль.
- Для вирішення цієї проблеми зручними є дещо розумніші ітератори, до прикладу:
  - здатні пропускати елементи – фільтрувати (модифікують, як себе веде операція типу ++);
  - здатні викликати якусь функцію при розіменуванні (модифікують, власне, розіменування).
- За їх допомогою можна створювати адаптери діапазонів (range adaptors) – об'єкти, що із діапазону створюють новий діапазон.
- Важливий їх різновид – view adaptor. Вони, не змінюючи вихідний діапазон, можуть не включати частину його елементів, відфільтровуючи "на льоту".
- Views є одночасно діапазонами та лінивими (lazy) алгоритмами над діапазонами, що можуть мати свій стан (stateful).
- Для негайних операцій існують також actions – дії (?).
  - Actions не входять в C++23.

Ranges-v3

- Подивимося на діапазони та адаптери на прикладі Ranges-v3. Це – референтна реалізація діапазонів із стандарту, хоча не все із неї ввійшло туди.
- Адреса: `https://github.com/ericniebler/range-v3`
- Документація: `https://ericniebler.github.io/range-v3/`
- Стаття від автора, про прийняття в стандарт: `http://ericniebler.com/2018/12/05/standard-ranges/`.
- Header-only. Для підключення можна або включити все:

  #include <range/v3/all.hpp>

  Або, для оптимізації часу компіляції та загального впорядкування, включити основне і підключати решту за потреби:

  #include <range/v3/core.hpp>
  #include <range/v3/....

◇ Як же воно довго компілюється... Наявність справжніх концепцій прискорює, але – не принципово.

◇ Зате std::range компілюється помітно швидше.

Тривіальне використання

- Алгоритми працюють із інтервалами. Однак, зазвичай, алгоритму передається цілий контейнер – при тому, все рівно, доводиться писати багато:

```
vector<int> v1{4, 3, 6, 1, 7, 9, 11, 0};
std::sort(v1.begin(), v1.end());
std::for_each( v1.begin(), v1.end(), [](auto x){ cout << x << ", "; });
cout << endl;
```

- Краще так:

```
ranges::sort( v1 );
ranges::for_each( v1, [](auto x){ cout << x << ", "; });
cout << endl;
```

- ranges::transform() – приклад **алгоритму** над діапазоном.
- Є також **адаптери** ranges::actions::transform() та ranges::views::transform() – "лінивий" варіант.
- ◇ Нижче уважно пильнуйте, із якою саме із цих сутностей маєте справу!
- Ремарка: в стандарті view було перейменовано у views, тому з часом бібліотека теж здійснила таке перейменування. Крім того, в стандарті є std::views – синонім до std::ranges::views.

Композиція

- Розглянемо типовий приклад – всі непарні числа із певного контейнера перетворити у стрічки:

```
std::vector<int> v2{1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
std::vector<int> temp2;
std::vector<string> res2;
std::copy_if(v2.begin(), v2.end(), std::back_inserter(temp2),
                                   [](auto x){ return x%2 == 1; } );
std::transform(temp2.begin(), temp2.end(), std::back_inserter(res2),
                                   [](auto x){ return std::to_string(x);} );
```

- Або так:

```
std::vector<int> v2{1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
auto res2r = v2
    | ranges::views::remove_if([](auto i){return i % 2 == 1;})
    | ranges::views::transform ([](auto i){return std::to_string(i);});
```

- Або ще простіше:

```
auto res3s = ranges::iota_view(2, 11)
    | ranges::views::stride(2)
    | ranges::views::transform ([](int i){return std::to_string(i);});
```

- Для комбінування використовується оператор |, по аналогії із конвеєрами (pipeline) bash та інших командних інтерпретаторів.

Композиція – стандарт

- В C++23 є views::stride(2), але, станом на 06.2023, не підтримується жодним компілятором.
- Однак, views::remove_if() – відсутній в C++23.
- ◇ Є лише ranges::remove_if() – якого немає в ranges-v3.
- Згідно авторів `https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p1899r1.html`, зробити із тим щось, в межах C++20, не особливо можна, оскільки такі реалізації матимуть невизначену поведінку (подробиці в тексті)

```
auto res3s1  = ranges::iota_view(2, 11)
             | views:: filter ([f = false](int i) mutable {return f = !f;})
             | views::transform ([](int i){return std::to_string(i);});
```

- Ремарка: коли ця презентація створювалася, підтримки компіляторами C++20 ще не було, тому вона розроблялася для ranges-v3. В 2022, при спробі портувати на стандарт, виявилося, що в стандарт не включено жоден адаптер із використаних в прикладах...

Нескінчені діапазони

- Підтримуються діапазони, нескінчені в теорії.
- Зрозуміло, що ця підтримка – "лінива", і є механізми обмежити реальну кількість.
- view::iota (ranges::iota_view) дозволяє – генерувати послідовності довільних інкрементабельних об'єктів[3].
- Аналог попередньої задачі, коли дані беруться не із контейнера, а генеруються на льоту:

```
auto res3r  = ranges::iota_view(1, 11)
    | ranges::views::remove_if([](int i){return i % 2 == 1;})
    | ranges::views::transform([](int i){return std::to_string(i);});
```

- виглядатиме так:

```
auto res4r  = ranges::views::iota(1, ranges::unreachable)
    | ranges::views::remove_if([](int i){return i % 2 == 1;})
    | ranges::views::transform([](int i){return std::to_string(i);})
    | ranges::views::take(5);
```

- Раніше замість views::iota() використовувався views::ints(). Був його варіант із з одним аргументом вважається застарілим, тепер використовується ranges::unreachable (ranges-v3) чи std::unreachable_sentinel.

[3] Про походження назви ф-ції iota(): https://stackoverflow.com/a/9244949.

Нескінчені діапазони – стандарт

- Оскільки в стандарті відсутній views::remove_if, замість:

```
auto res4r  = ranges::views::iota(1, ranges::unreachable)
    | ranges::views::remove_if([](int i){return i % 2 == 1;})
    | ranges::views::transform([](int i){return std::to_string(i);})
    | ranges::views::take(5);
```

- доведеться писати:

```
auto res4r = views::iota(1, std::unreachable_sentinel)
    | views::filter([](int i){return i % 2 != 1;})
    | views::transform([](int i){return std::to_string(i);})
    | views::take(5);
```

- А ще можна так:

```
auto res5inf  = ranges::views:: ints(1, ranges:: unreachable)
    | ranges:: views:: remove_if([](int i){return i % 2 == 1;})
    | ranges:: views:: transform ([](int i){return std:: to_string(i );});

auto res5r = res5inf | ranges:: views:: take(5);
```

- Стандартний варіант:

```
auto res5inf1  = views:: iota(1, std:: unreachable_sentinel)
    | views:: filter ([](int i){return i % 2 != 1;})
    | views:: transform ([](int i){return std:: to_string(i );});

auto res5r1 = res5inf1 | views:: take(5);
```

- **Увага!** view::transform повертає об'єкт класу (ranges::transform_view), що є нащадком ranges::view_interface. Останній має методи виду begin(), end(), front(), back() і т.д., але метод size() має лише, якщо його можна обчислити за константний час. Тому, для коду вище, щоб взнати розмір результату, слід використати ranges::distance(res4r).

- Ще приклад, від автора бібліотеки – сума квадратів перших 10 цілих чисел:

```
int sum = ranges::accumulate(views::iota(1, ranges::unreachable)
                           | view::transform([](int i){return i*i;})
                           | view::take(5), 0);
```

- В C++23 додали std::ranges::fold_left замість accumulate:

```
int sum = ranges::fold_left( views::iota(1, std::unreachable_sentinel)
                           | views::transform([](int i){return i*i;})
                           | views::take(5),
                             0, std::plus<int>());
```

◇ Назву обрали таку, щоб не було асоціації лише з підсумовуванням.

◇ Є також fold_left_first – аналог, який в ролі першого елементу бере перший елемент діапазону, fold_right – з асоціативністю справа наліво; fold_right_last, fold_left_with_iter, fold_left_first_with_iter.

- В стандарті C++20 ranges::accumulate відсутній, тому доведеться робити якесь таке збочення:

```
int sum4r1 = 0;
        ranges::for_each(views::iota(1, std::unreachable_sentinel)
                           | views::transform([](int i){return i*i;})
                           | views::take(5), [&sum4r1](auto arg){ sum4r1 += arg; });
```

Не робіть так, якщо є можливість уникнути.

Actions

- Якщо контейнер слід модифікувати на місці, можна зробити щось таке:

```
vector<int> rv() {     .....     }
    .....

vector<int> v3_u { rv() | ranges::actions::sort
                        | ranges::actions::unique } ;
print_container(v3_u);
```

- Якщо контейнер вже існує:

```
v4a = std::move(v4a) | ranges::actions::sort | ranges::actions::unique;
```

Зверніть увагу на використання move – на вхід передаємо лише rvalue reference.

- Те ж можна зробити так:

```
v4b |= ranges::actions::sort | ranges::actions::unique;
```

- Або з допомогою синтаксису виклику функцій:

```
ranges::actions::unique(ranges::actions::sort(v4c));
```

- Actions в стандарті C++23 відсутні.

- Якщо я захочу їх у зворотному порядку, можна зробити так:

```
v4b |= ranges :: actions :: sort  | ranges :: actions :: unique
     | ranges :: actions :: reverse ;
```

- А так – після цього відкинути два найбільших та два найменших:

```
v4b |=   ranges :: actions :: sort  | ranges :: actions :: unique
     | ranges :: actions :: reverse
     | ranges :: actions :: slice (2, ranges :: end − 2);
```

split

- view::split() приймає роздільник, як символ чи інтервал, view::split_when() предикат, який отримує один елемент із діапазону і має сказати, чи він є роздільником, і ріжуть інтервал на (увага) інтервал інтервалів.

```
string  text2     {.....................};
vector<string> words2a = text2 | ranges::views::split(' ');
```

В прикладах часто пишуть так, щоб порахувати кількість слів:

```
ranges::distance( text2 | ranges::views::split(' ') )
```

Або й так, отримати довжину кожного слова:

```
text2 | ranges::views::split(' ')
      | ranges::views::transform(ranges::distance);
```

Хоча це більше навчальний приклад – на практиці може бути неефективним.

- Сепаратор може мати більше за один символ, але для цього слід передавати діапазон. Зокрема, для C-стрічок:

```
vector<string> words3 = text3 | ranges::views::split(
                        ranges::views::c_str("long_separator"));
```

- Однак, не слід забувати, що повертається саме range! Це буде аж ніяк не вектор стрічок:

  auto words2c = text2 | ranges::views::split(' ');

  Тип words2c для GCC буде якимось таким[4]:

```
ranges::split_view<ranges::ref_view<std::string>,
       ranges::single_view<char> >
```

  Навіть to_vector не допомагає:

  auto words2b = text2 | ranges::views::split(' ') | ranges::to<vector>;

  дасть:

```
std::vector<ranges::detail::split_outer_iterator<
    ranges::split_view<ranges::ref_view<std::string>,
        ranges::single_view<char> >, true>::value_type,
    std::allocator<ranges::detail::split_outer_iterator<
        ranges::split_view<ranges::ref_view<std::string >,
        ranges::single_view<char> >, true>::value_type> >
```

- Зате таке – теж спрацює:

  vector<vector<**char**>> words2d = text2 | ranges::views::split(' ');

[4] Тут такий жах: std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > > згорнуто в std::string.

join

- Можна і склеювати:

```
vector<string> words4 =     ...............;
string  text4  = words4 | ranges::move | ranges::actions::join ;
```

- move потрібен, оскільки ranges::actions::join працюють лише із rvalue reference.

Перетворення в string_view

- Цікавий приклад звідси: `https://stackoverflow.com/a/48403210`

```
auto sview = text2
           | ranges::views::split(' ')
           | ranges::views::transform([](auto &&rng) {
    return std::string_view(&*rng.begin(), ranges::distance(rng));
});
```

Projections

- Багато які алгоритми приймають ще так звану projection – перш ніж із результатом розіменовування ітератора, щось буде робитися, до нього застосовується передана функція.
- До прикладу:

```
template<RandomAccessIterator I, Sentinel<I> S,
         class Comp = ranges::less<>, class Proj = ranges:: identity >
  requires Sortable<I, Comp, Proj>
I sort( I first , S last , Comp comp = Comp{}, Proj proj = Proj{} );

template<InputIterator I , Sentinel<I> S, class Proj = ranges:: identity ,
         IndirectUnaryInvocable<projected<I, Proj>> Fun >
ranges::tagged_pair<tag::in(I), tag::fun(Fun)>
  for_each(I first , S last , Fun f, Proj proj = Proj{});
```

Див. аргумент proj.

Приклади

- Приклади на наступних слайдах взяті, іноді – із змінами, звідси: `https://www.walletfox.com/course/examples_range_v3.php` та `https://www.walletfox.com/course/quickref_range_v3.php`.
- Перетворення із двійкової системи в десяткову:

```
auto const bin1 = {1u,1u,1u,0u};
//const vector<int> bin1 = {1u,1u,1u,0u}; // <== Або й так
auto v_rev1 = bin1 | ranges::views::reverse; // [0,1,1,1]
auto v_bin1 = ranges::views::generate(
    [n = 1u] () mutable {return (n <<= 1u) >> 1u;}); // [1,2,4,8...]
auto val1 = ranges::inner_product(v_rev1, v_bin1, 0u);
                                    // 0*1+1*2+1*4+1*8 = 14
```

- "snake_case to CamelCase":

```
auto s = std::string{"feel_the_force"};
auto words = s | ranges::views::split('_'); // [[f,e,e,l],[t,h,e],[f,o,r,c,e]]
auto words_cap = words | ranges::views::transform([](auto w){
        auto head = w
          | ranges::views::take(1)
          | ranges::views::transform([](unsigned char c){
                        return std::toupper(c);}); // e.g. [F]
    return ranges::views::concat(head, w | ranges::views::tail);

  }); // [[F,e,e,l],[T,h,e],[F,o,r,c,e]]
std::string s_camelcase = words_cap | ranges::views::join; // FeelTheForce
```

- Такий конвертер навіть можна собі зберегти:

```
auto snake_to_camel = ranges::views::split('_')
    | ranges::views::transform([](auto w){
        auto head = w | ranges::views::take(1)
                      | ranges::views::transform([](unsigned char c)
                        {return std::toupper(c);});
        return ranges::views::concat(head, w | ranges::views::tail);
    }) | ranges::views::join;

    std::string s2 = s | snake_to_camel;
    cout << std::string{s | snake_to_camel} << endl;
```

- Більше – в першоджерелі,
  `https://www.walletfox.com/course/examples_range_v3.php`

Потенційна проблема продуктивності

- Окрім швидкості компіляції є ще одна складність. Конвеєр, обробляючи діапазони, іноді змушений проходити по них декілька раз. Див., наприклад: `https://www.reddit.com/r/cpp/comments/8ytrnb/what_i_dont_like_about_ranges/`.

Інвалідація

- Будь-яка операція, що інвалідує вхідних діапазон – його ітератори чи "охоронці"-sentinels, інвалідує і всі view, що на нього посилаються.
- Деякі відображення, типу view::filter, інвалідуються за будь-яких змін вхідного діапазону.

Константність

- Хоча view може являти собою лише пару ітераторів, деякі відображення, для підвищення ефективності, можуть кешувати щось.
- У такому випадку, view не буде мати константних begin() та end().
- З іншого боку, якщо він має константні begin() і end(), він нічого не кешує, і їх результати є константними – а значить, потокобезпечними.
- Це, звичайно, як і для інших видів ітераторів/вказівників, не стосується даних, на які вони вказують.
- Автор рекомендує використовувати не-константні begin() та end(), там, де це можливо, а для потокобезпечності – копіювати view у відповідні потоки.

Створення власних view

- Як показано вище, на прикладі `snake_to_camel`, найпростіший спосіб створити свій view – комбінувати інші.
- Якщо ж писати свої, слід врахувати наступні подробиці про структуру бібліотеки.
- Нехай ми створюємо view із іменем XXX.
- Тоді, повинен існувати шаблон view_XXX, що задовільняє вимогам концепції View[5] та InputRange[6] або потужнішого.
- Адаптер, із іменем XXX_fn із перевантаженими операторами operator() і operator|.
- Екземпляр адаптера, із яким і працюватиме користувач: view::XXX.
- Реалізація з нуля описана тут: `https://hannes.hauswedell.net/post/2018/04/11/view1/`, код звідти – див. `Projects/l32_ranges_v3_demo/view_add_constant.cpp`.
- Також про створення власних діапазонів, адаптерів та ітераторів, розповідається в офіційній (трохи застарілій і неповній) документації: `https://ericniebler.github.io/range-v3/`, в розділі "Create Custom Ranges" і далі.
- Підхід для стандартної бібліотеки потребує окремого розгляду.
- ◇ Приклад наведено тут: `https://mariusbancila.ro/blog/2020/06/06/a-custom-cpp20-range-view/`.

[5] `https://en.cppreference.com/w/cpp/ranges/view`.

[6] `https://en.cppreference.com/w/cpp/ranges/input_range`.

Ніблоїди (DRAFT)

- Niebloid – об'єкт-функція, який забороняє ADL.
- Ціль – щоб замість алгоритмів з std::, підхоплювало за потреби алгоритми з std::ranges::.
- Детальніше – див. цю відповідь: `https://stackoverflow.com/a/62929027`.

Iterator sentinels

- Концепція використовувати спеціальні ітератори іншого типу в ролі/на заміну v.end(), цікава сама по собі, безвідносно бібліотеки ranges.
- Ідея – створити відповідний клас із перевантаженими операторами порівняння:

```
class iterator { ..... };

bool operator==(iterator lhs, iterator rhs); // Звичні
bool operator!=(iterator lhs, iterator rhs);

class sentinel {};

bool operator==(iterator iter, sentinel ); // І спеціалізовані.
bool operator!=(iterator iter, sentinel );

bool operator==(sentinel, iterator iter );
bool operator!=(sentinel, iterator iter );
```

- Приклади взято із статті: `https://foonathan.net/2020/03/iterator-sentinel/`.

Використання Iterator sentinels для C-стрічок

- ```
  class zstring_sentinel {};
  bool operator==(const char* str, zstring_sentinel ) { return *str == '\0'; }
  bool operator!=(const char* str, zstring_sentinel ) { return *str != '\0'; }

  struct zstring_range {
      const char* str;
      zstring_range(const char* str_): str(str_){}
      auto begin() const { return str; }
      auto end() const { return zstring_sentinel {}; }
  };

      char str [] = "abcd";
      for(auto x: zstring_range{str}){
          std::cout << x << ",";
      }
  ```

- Приклади взято із статті: `https://foonathan.net/2020/03/iterator-sentinel/`. На жаль, авторський приклад із ranges змусити працювати не вдалося.
- Код покладається на range for із C++17, із різними типами для begin та end ітераторів: `https://en.cppreference.com/w/cpp/language/range-for`.
  ```

std::unreachable_sentinel (C++20)

- std::unreachable_sentinel – спосіб вказувати верхню межу для необмеженого інтервалу.
- Має перевизначений оператор operator==, який може порівнювати із будь-яким weakly_incrementable типом і завжди повертає false.
- Див. також `https://en.cppreference.com/w/cpp/iterator/unreachable_sentinel_t`.
- std::weakly_incrementable – концепція типів, що мають ++i та i++, які мають, грубо кажучи, однаковий сенс та не змінюють адресу самого об'єкта i, однак, якщо a == b, не гарантується, що ++a == ++b.
- `https://en.cppreference.com/w/cpp/iterator/weakly_incrementable`

Стандартизація – 1

- Авторський огляд, що прийнято в стандарт – ще до його затвердження: `http://ericniebler.com/2018/12/05/standard-ranges/`
- Оскільки більшість засобів із ranges-v3 не потрапили в стандарт, маємо такий план: `https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2214r0.html` – "A Plan for C++23 Ranges".
- Деякі сутності називаються по іншому, наприклад, замість:

  `ranges::views::ints(1, ranges::unreachable)`

  пишемо:

  `views::iota(1, std::unreachable_sentinel)`

  простір імен view перейменовано в views, а для `std::ranges::view` є синонім `std::views`.
- Огляд того, що потрапило в C++20: `https://www.walletfox.com/course/quickref_range_v3.php`.
- Ще один огляд: `https://itnext.io/c-20-ranges-complete-guide-4d26e3511db0`.
- Стандартні view: `https://en.cppreference.com/w/cpp/ranges`.
- Стандартні алгоритми над range: `https://en.cppreference.com/w/cpp/algorithm/ranges`.

Стандартизація – 2

- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2214r0.html#stdaccumulate-rangesfold – "A Plan for C++23 Ranges", 2020-10-14.
- https://mariusbancila.ro/blog/2022/12/23/the-cpp23-standard-break-down/ – містить список доданого в C++23. Нарешті ranges::to...
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2760r0.html – "A Plan for C++26 Ranges".

Підтримка компіляторів – 2023

- Підтримуються достатньо новими GCC (11-13), Clang (16-17), MSVC 2022.
- ◇ На відміну від 2019 року, якихось спеціальних опцій компілятору, окрім версії стандарту – C++23[7] не потрібно.
- В цілому, підтримка в MSVC краща – див. у прикладі l32_ranges_std_demo приклади, обгорнуті "#ifdef _MSC_VER".
- ◇ Зокрема, станом на 06-2023, в GCC та Clang відсутня підтримка ranges::to та ranges::fold_left.
- Актуальна інформація щодо підтримки: `https://en.cppreference.com/w/cpp/compiler_support#C.2B.2B23_library_features`.
- Підтримка в GCC: `https://gcc.gnu.org/wiki/LibstdcxxTodo`.

[7] В CMake – set(CMAKE_CXX_STANDARD 23).

Підтримка компіляторів – 2019

- **Інформація застаріла, залишена для історичної довідки.**
- Підтримуються достатньо новими GCC, Clang, MSVC 2017.
- Якщо компілятор підтримує концепції – краще дозволити.
- MSVC 2017 потребує (станом на липень 2019) ряд ключів: "/std:c++17 /permissive- /experimental:preprocessor /wd5105", 2019 – варто замінити "/experimental:preprocessor" на "/Zc:preprocessor", та й /std на c++20 або c++latest.
- Фрагмент CMakeLists.txt:

```
set(CMAKE_CXX_STANDARD 17)

if(CMAKE_COMPILER_IS_GNUCC OR CMAKE_COMPILER_IS_GNUCXX)
    set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -fconcepts")
elseif(MSVC)
    set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} /std:c++17 \
     /permissive- /Zc:preprocessor /wd5105")
endif()
```

Посилання – 1

- `https://arne-mertz.de/2017/01/ranges-stl-next-level/` – загальний огляд.
- `https://www.fluentcpp.com/2018/02/09/introduction-ranges-library/` – ще один, від автора попереднього тексту.
- `https://www.modernescpp.com/index.php/the-new-ranges-library` – і від іншого автора.
- Приклади використання: `https://mariusbancila.ro/blog/2019/01/20/cpp-code-samples-before-and-after-ranges/`, і переклад російською: `https://habr.com/ru/company/otus/blog/456452/`.
- Чернетка пропозиції до стандарту, 2018-11-09: `http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0896r4.pdf`
- Документація, часто актуальніша, ніж на офіційному сайті (сам автор зізнається, що там вона застаріла трохи): `https://www.walletfox.com/course/quickref_range_v3.php`
- Приклади із попереднього сайту: `https://www.walletfox.com/course/examples_range_v3.php`.

Посилання – 2

- Просто приклад: `https://github.com/nsergey82/ranges-couroutines`
- Авторський огляд, що ввійшло в стандарт: `http://ericniebler.com/2018/12/05/standard-ranges/`.
- `https://habr.com/ru/post/481402/` – "Ленивая реализация обхода дерева дочерних элементов класса QObject", з використанням ranges.
- `https://devblogs.microsoft.com/cppblog/c20-ranges-are-complete-in-visual-studio-2019-version-16-10/`
- `https://www.cppstories.com/2022/ranges-composition/`
- `https://mariusbancila.ro/blog/2023/01/16/how-to-make-chunks-of-a-range-in-cpp23/` – створення підпослідовностей, з зсувом на 1-N (1, 3, 4, 5, 6, 7 –> 1, 2, 3; 2, 3, 4; 3, 4, 5; ...), на розмір підпослідовності, (–> 1, 2, 3; 4, 5, 6; 7), згідно предикату.
- `https://devblogs.microsoft.com/cppblog/documentation-for-cpp20-ranges/` – спроба документування від MSVC.
- `https://quuxplusone.github.io/blog/2023/08/13/non-const-iterable-ranges/` – "Some C++20 ranges aren't const-iterable".
- `https://biowpn.github.io/bioweapon/2023/09/27/k-way-merge.html` – "Implement K-Way Merge with std::ranges"

Посилання – 3

- "When ranges::for_each behaves differently from for" – есей на тему.
- C++23 додав підтримку користувацьких адаптерів: "Pipe support for user-defined range adaptors".
- "Rvalue Ranges and Views in C++20" – Borrowed range, це такий, ітератори якого не "провиснуть", якщо припинить існувати сам об'єкт. Наприклад, std::string_view чи std::span є "позиченими", а std::string чи std::vector – ні. Такий об'єкт можна перетворити у view за допомогою views::all(). Адаптери працюють лише над lvalue і Borrowed range rvalue. Повідомити, що певний інтервал є "позиченим" можна так:

```
template <>
inline constexpr bool
  std::ranges::enable_borrowed_range<my::StringRef> = true;
```

- "How to join or concat ranges, C++26".
- "Using Token Sequences to Iterate Ranges" – есей про проблеми продуктивності інтервалів, і як їх можна вирішувати.
- "Views as Data Members for Custom Iterators".
- "How to Split Ranges in C++23 and C++26".
- "The Road to Flux 1.0" – дуже цікава альтернатива до std::ranges, в розробці, з надією на C++29.
- "PSA: views::single doesn't really view" – як зробити view з std::pair.

